

Sistem Smart Home Integrat: Tracker Solar și Barieră de Acces Automatizată

AUTOR: Busioc Crinu Marian

COORDONATOR: Șef lucrări Dr. Ing. Laura Mihaela Lelutiu

Rezumat: Acest proiect prezintă dezvoltarea unui sistem inteligent dual, conceput pentru eficientizarea captării de energie și automatizarea securității. Ansamblul este format din două componente principale care funcționează simultan:

1. Un sistem de orientare a panourilor solare (tracker solar cu două axe), capabil să urmărească automat cea mai puternică sursă de lumină pentru a maximiza randamentul energetic.
2. O barieră de acces automatizată, dotată cu senzori de mișcare pentru deschidere și senzori ultrasonici de siguranță pentru prevenirea coliziunilor. Întregul sistem este monitorizat și controlat printr-o aplicație mobilă Android personalizată, comunicând prin modulul Bluetooth. Aplicația oferă telemetrie în timp real (citirea tensiunii bateriei), comutare între modurile de funcționare (Automat/Manual), control din joystick virtual și integrarea unui asistent pentru comenzi vocale.

Cuvinte cheie: IoT, Arduino, tracker solar, barieră automată, Bluetooth, telemetrie, MIT App Inventor.

1. INTRODUCERE

În contextul actual al dezvoltării tehnologice, integrarea surselor de energie regenerabilă și automatizarea locuințelor (Smart Home) au devenit priorități esențiale pentru un stil de viață sustenabil. Panourile solare fixe au un randament limitat pe parcursul zilei, iar sistemele de acces standard oferă adesea o flexibilitate redusă în operare.

Atunci când se dorește implementarea unui astfel de sistem inteligent, apare o alegere importantă: achiziționarea unor echipamente comerciale gata asamblate sau construirea unui sistem propriu (DIY), adaptat cerințelor specifice. Diferențele dintre aceste două opțiuni sunt semnificative și se reflectă în costurile implicate, posibilitățile de personalizare și gradul de implicare tehnică. În timp ce un sistem cumpărat oferă o instalare simplă, un ecosistem DIY bazat pe microcontrollere (precum Arduino) aduce satisfacția unui produs complet personalizat, cu performanțe optimizate și fără limitările aplicațiilor comerciale închise.

Proiectul de față explorează tocmai această abordare DIY, propunând o soluție IoT (Internet of Things) complexă. Scopul principal este interconectarea unui mecanism de producere a energiei verzi cu un sistem de securitate fizică, ambele aflându-se sub controlul direct al utilizatorului.

Spre deosebire de automatizările clasice, acest proiect pune un accent deosebit pe interfața om-mașină (HMI). Prin intermediul unei aplicații mobile dezvoltate de la zero, utilizatorul nu doar că poate acționa bariera sau mișca panourile solare, dar primește și feedback vizual constant

(telemetrie privind voltajul) și poate interacționa cu sistemul prin intermediul comenzilor vocale procesate în timp real. Astfel, lucrarea demonstrează cum componente electronice accesibile pot fi integrate într-o arhitectură software robustă pentru a rezolva probleme reale de eficiență și control.

2. ARHITECTURA HARDWARE A SISTEMULUI

Pentru realizarea acestui proiect, s-a optat pentru o arhitectură hardware complexă și robustă, capabilă să gestioneze simultan multiple semnale de control, dar și să asigure un management eficient și sigur al energiei. Componentele au fost structurate în cinci module principale.

2.1. Unitatea Centrală de Procesare

Microcontroller-ul Arduino Mega 2560: Reprezintă „creierul” sistemului. A fost ales în defavoarea variantei Uno datorită numărului extins de pini I/O și a prezenței porturilor Serial hardware multiple (ex. *Serial1* utilizat pentru comunicarea Bluetooth fără a interfera cu portul principal). Acesta procesează logica sistemului anti-oscilație al panourilor, monitorizează senzorii de siguranță și controlează afișajele.

2.2. Managementul Energiei și Alimentarea (Power Management) Acest subsistem asigură funcționarea autonomă și în siguranță a întregului ansamblu:

2x Baterii Li-Ion și Suport: Reprezintă sursa principală de stocare a energiei.

Modul BMS (Battery Management System): O componentă critică pentru siguranță. Asigură încărcarea echilibrată a celulelor Li-Ion și le protejează împotriva supra-descărcării, supraîncărcării și scurtcircuitelor, prelungind durata de viață a acumulatorilor.

Modul Variator Coborâtor (Buck Converter): Având în vedere că bateriile furnizează o tensiune mai mare decât nivelul logic de 5V necesar plăcii și motoarelor, modulul Buck este folosit pentru a coborî eficient tensiunea, minimizând pierderile sub formă de căldură (spre deosebire de un regulator liniar).

2x Condensatoare de filtraj (470 μ F): Sunt adăugate pe liniile de alimentare ale servomotoarelor pentru a absorbi vârfurile de consum (spikes). Previn astfel căderile bruște de tensiune (brown-outs) care ar putea reseta microcontroller-ul în momentul acționării motoarelor.

Buton ON/OFF: Comutator principal pentru întreruperea circuitului general.

2.3. Sistemul de Captare a Energiei și Telemetrie

2x Panouri Fotovoltaice (conectate în serie): Conexiunea în serie a fost aleasă pentru a însuma tensiunea furnizată de cele două panouri, obținându-se astfel o tensiune superioară. Această abordare asigură o încărcare mai eficientă a sistemului de baterii și o acoperire optimă a necesarului energetic pentru componentele hardware.

Diodă de protecție (Blocking Diode): Montată pe circuitul panourilor solare pentru a permite trecerea curentului într-un singur sens (de la panouri spre baterie) și a bloca descărcarea bateriei înapoi în panouri pe timpul nopții.

Senzor de Tensiune (Voltaj) și Modul Releu: Senzorul măsoară nivelul de tensiune. Pentru a nu folosi doi senzori separați, un modul Releu este acționat inteligent pentru a comuta circuitul senzorului între bornele Bateriei și cele ale Panoului Solar.

Senzor Touch (Capacitiv): Este interfața fizică prin care utilizatorul dictează releului să schimbe sursa de citire a tensiunii afișată pe ecran și în aplicație.

2.4. Sistemul de Senzori și Comunicație (Intrări)

Modulul Bluetooth (HC-05/06): Asigură puntea de comunicație wireless bidirecțională între Arduino și aplicația mobilă Android (recepționează comenzi și trimite telemetria).

Senzorul Ultrasonic (HC-SR04): Măsoară timpul de zbor al undelor sonore pentru a detecta prezența unui obstacol (ex. o mașină) sub brațul barierei, acționând ca element principal de siguranță.

Senzorul PIR (HC-SR501): Detectează variațiile de radiație infraroșie generate de mișcare pentru a sesiza apropierea unei mașini, având rolul de a declanșa automat deschiderea barierei pentru vehiculele care doresc să iasă din parcare.

4x Fotorezistențe (LDR) cu 4x Rezistențe (10 k Ω): Formează divizoare de tensiune dispuse în cruce, permițând microcontroller-ului să analizeze sursa de lumină maximă pentru orientarea tracker-ului.

Modul Joystick: Permite ghidarea manuală a panourilor pe axele X și Y, util pentru mentenanță sau calibrare.

2.5. Actuatori și Interfață Locală (Ieșiri)

2x Servomotoare MG90S (Metal Gear): Utilizate pentru mecanismul Pan-Tilt al panourilor fotovoltaice. Au fost preferate modelele cu angrenaj metalic (MG) deoarece suportă o greutate mecanică (cuplu) mai mare comparativ cu cele din plastic, asigurând stabilitatea structurii la vânt.

1x Servomotor SG90: Acționează brațul barierei de acces, o sarcină mecanică ușoară pentru care angrenajele standard sunt suficiente.

Display OLED: Oferă o interfață vizuală locală modernă (HMI), afișând starea sistemului, nivelurile de tensiune sau modul de funcționare curent.

Buzzer și LED (Indicator Manual): Oferă feedback direct utilizatorului. Buzzer-ul emite alarme sonore în caz de eroare la închiderea barierei, iar LED-ul semnalizează activarea modului de control Manual al panourilor din joystick.

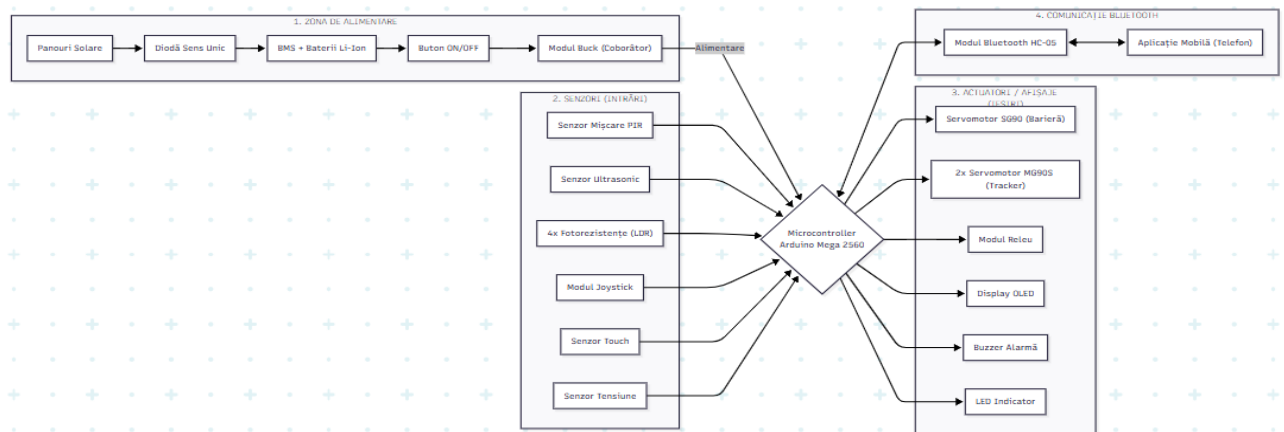


Fig. 1. Asamblarea hardware a sistemului (sau Schema Bloc a sistemului).

3. ARHITECTURA SOFTWARE ȘI ALGORITMI DE CONTROL

Componenta software a acestui proiect este împărțită în două entități care comunică în permanență: codul rulat local de microcontroller (C/C++ adaptat pentru Arduino) și aplicația mobilă dezvoltată în mediul vizual MIT App Inventor.

3.1. Logica Microcontroller-ului (Arduino Mega 2560)

Programul principal a fost optimizat pentru a rula fără întreruperi blocante (non-blocking architecture). În loc de funcția standard `delay()`, care ar "îngheța" procesorul și ar face sistemul surd la comenzile Bluetooth sau la senzorii de siguranță, s-a utilizat funcția `millis()`. Aceasta permite multitasking-ul: bariera poate coborî în timp ce panourile se mișcă, iar voltajul este trimis la fiecare 2 secunde.

Pentru mișcarea panourilor fotovoltaice, s-a implementat un algoritm cu histerezis (zonă moartă/toleranță de 25 de unități). Microcontroller-ul citește cele 4 fotorezistențe, calculează media pe axe și mișcă servomotoarele cu pași fini (2 grade la fiecare 50 ms). Toleranța previne fenomenul de „jitter” (tremur constant al motoarelor atunci când lumina este uniformă).

3.2. Aplicația Mobilă (MIT App Inventor)

Aplicația Android a fost concepută având la bază interacțiunea intuitivă (HMI - Human Machine Interface). Funcțiile software principale includ:

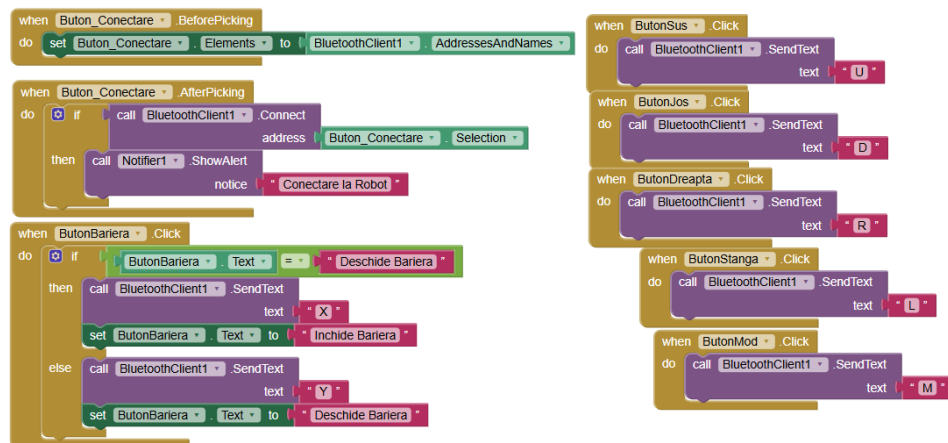


Fig. 2. Interfața grafică (GUI) a aplicației mobile.

Parsarea Datelor de Telemetrie: Aplicația folosește blocul logic ReceiveText pentru a capta șirul de caractere trimis prin Bluetooth. Pentru a face distincția între tensiunea panoului și cea a bateriei, s-a creat un filtru care caută prefixele "P" sau "B", le elimină, și afișează strict valoarea numerică pe ecranul utilizatorului.

Controlul Inteligent al Butoanelor (Toggle): Butonul de acționare al barierei își modifică dinamic textul și funcția („Deschide” devine „Închide” după apăsare), expediind caracterele de control "X" sau "Y".

Recunoașterea Vocală Avansată: Pentru a elimina erorile de procesare generate de asistentul Google (ex. spații goale adăugate automat sau conjugări), algoritmul transformă toată propoziția în litere mici (downcase) și caută doar un fragment specific (ex. contains text piece "nchide"). Astfel, bariera răspunde impecabil indiferent de formularea exactă a utilizatorului.



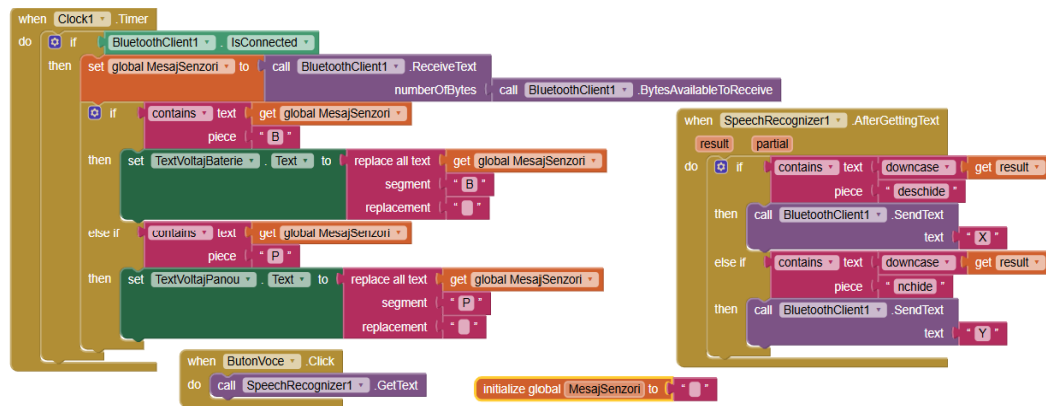


Fig. 3. Secvență de cod vizual - blocurile logice din MIT App Inventor.

3.3. Formule și Calcule Matematice Utilizate

Pentru a asigura funcționarea corectă a senzorilor, logica software se bazează pe transformarea semnalelor brute în mărimi fizice reale, utilizând următoarele formule matematice:

Pentru măsurarea distanței de siguranță cu senzorul ultrasonic HC-SR04, s-a folosit formula mișcării rectilinii uniforme, adaptată și împărțită la 2 pentru a compensa timpul de zbor al ecoului (traseul dus-întors al sunetului):

(1)

$$d = \frac{t \times 0.034}{2}$$

unde:

d = distanța măsurată până la obstacol (cm),

t = timpul de zbor al impulsului sonor (μs),

0.034 = viteza de propagare a sunetului în aer (cm/ μs).

Pentru conversia valorii analogice citite de pe pinul senzorului de tensiune într-o valoare reală (Volți), s-a utilizat rezoluția pe 10 biți a convertorului Analog-Digital (ADC) integrat în Arduino, adăugând în cod și o variabilă software de corecție:

(2)

$$V = \frac{Valoare_{ADC} \times 5.0}{1024} \times FactorCalibrare$$

unde:

V = Tensiunea calculată și transmisă către aplicație (V),

$Valoare_{ADC}$ = Valoarea digitală brută citită pe pinul analogic (între 0 și 1023),

5.0 = Tensiunea de referință a microcontroller-ului (V)

FactorCalibrare=O constantă de ajustare (ex. 4.95) utilizată pentru a corecta precizia hardware a modului senzor.

Pentru orientarea panourilor fotovoltaice pe axa verticală, algoritmul calculează diferența absolută dintre mediile aritmetice ale senzorilor. Mișcarea se execută doar dacă această eroare depășește toleranța stabilită:

(3)

$$\Delta_E = |Media_{sus} - Media_{jos}|$$

unde:

Δ_E = Diferența absolută de iluminare (Eroarea de poziționare),

$Media_{sus}$ = Media aritmetică a valorilor celor două fotorezistențe superioare,

$Media_{jos}$ = Media aritmetică a valorilor celor două fotorezistențe inferioare.

Pentru determinarea nivelului de iluminare pe fiecare zonă a tracker-ului solar, sistemul calculează media aritmetică a senzorilor LDR corespunzători fiecărei extremități. De exemplu, pentru zona superioară, se utilizează următoarea formulă:

(4)

$$Media_{sus} = \frac{LDR_{stanga_sus} + LDR_{dreapta_sus}}{2}$$

unde:

$Media_{sus}$ = Media aritmetică a nivelului de lumină pentru partea superioară a panoului,

LDR_{stanga_sus} = Valoarea analogică brută citită de la fotorezistența din stânga-sus,

$LDR_{dreapta_sus}$ = Valoarea analogică brută citită de la fotorezistența din dreapta-sus.

Similar orientării pe axa verticală, pentru orientarea panourilor fotovoltaice pe axa orizontală (stânga-dreapta), algoritmul calculează diferența dintre mediile laterale. Mișcarea se execută doar dacă această eroare depășește toleranța stabilită:

(5)

$$\Delta E_{orizontal} = Media_{stanga} - Media_{dreapta}$$

unde:

$\Delta E_{orizontal}$ = Diferența de iluminare pe axa orizontală (Eroarea de poziționare laterală),

$Media_{stanga}$ = Media aritmetică a valorilor celor două fotorezistențe din partea stângă,

$Media_{dreapta}$ = Media aritmetică a valorilor celor două fotorezistențe din partea dreaptă.

4. STUDIU DE CAZ ȘI METODOLOGIA DE TESTARE

Pentru a valida funcționarea corectă a ansamblului, sistemul a fost supus unei serii de teste practice, urmărindu-se timpii de răspuns, precizia mecanică și stabilitatea conexiunilor. Probele s-au desfășurat simulând condiții reale de utilizare.

4.1. Testarea Mecanismului de Orientare Solară Pentru a verifica eficiența tracker-ului solar pe axele Pan-Tilt, s-au folosit surse de lumină artificială direcționale (lanterne), simulând mișcarea soarelui. Sistemul a demonstrat o capacitate excelentă de a urmări sursa de lumină, orientând panourile fotovoltaice în unghiul optim. Sensibilitatea senzorilor LDR permite detectarea modificărilor de iluminare chiar și la variații mici, asigurând o captare maximă a energiei.

4.2. Testarea Sistemelor de Acces și Siguranță Bariera automată a fost testată folosind obstacole fizice aflate în mișcare. Senzorul PIR a detectat constant prezența în zona de acces, inițiind secvența de deschidere. Funcția critică de siguranță – oprirea de urgență – a fost testată plasând un obiect sub brațul barierei în timpul coborârii. Senzorul ultrasonic a reacționat instantaneu, procesorul oprind servomotorul SG90 și activând alarma acustică (Buzzer-ul) fără nicio întârziere sesizabilă.

4.3. Testarea Comunicației Wireless și a Telemetriei S-a verificat raza de acțiune și stabilitatea modulului Bluetooth HC-05. Comunicația cu aplicația mobilă s-a menținut stabilă pe o rază de aproximativ 10 metri, fără pierderi de pachete de date. Afișajul pe display-ul OLED și actualizarea tensiunii pe telefon s-au realizat simultan, demonstrând că bucla principală de program a microcontroller-ului Arduino Mega nu suferă de blocaje (lag). Sistemul răspunde prompt atât la comenzile manuale din Joystick, cât și la comenzile vocale procesate de aplicație.

5. PROBLEME ÎNTÂMPINATE ȘI SOLUȚII IMPLEMENTATE

Pe parcursul dezvoltării și asamblării sistemului, au apărut o serie de provocări tehnice, atât la nivel fizic (hardware), cât și la nivel de codificare (software). Identificarea și rezolvarea acestora au reprezentat o parte esențială a procesului de inginerie a proiectului.

5.1. Conectarea incorectă a etajelor de alimentare (BMS și Buck)

Problema: Inițial, circuitul de alimentare a fost cablat eronat: bateriile Li-Ion au fost conectate mai întâi la variatorul de tensiune (Buck Converter) și abia apoi la sistemul de protecție (BMS). Această configurație a lăsat celulele expuse riscului de supradescărcare prin consumul continuu al modului Buck, nefiind protejate direct de BMS.

Soluția: S-a refăcut complet logica schemei de alimentare. Bateriile au fost conectate direct la bornele dedicate ale modului BMS, asigurând protecția la nivel de celulă. Modulul Buck a fost mutat pe ramura de ieșire (după BMS), coborând tensiunea abia după ce energia a trecut prin filtrul de siguranță.

5.2. Limitarea mecanică și protecția cablajelor (Pan-Tilt)

Problema: La primele teste ale tracker-ului solar, servomotoarele se roteau peste unghiul de siguranță al structurii. Această cursă exagerată ducea la tensionarea extremă a firelor de conexiune (existând riscul smulgerii lor) și la forțarea fizică a stâlpului de susținere al panourilor.

Soluția: S-au implementat limitări software (funcția de *constrain*) pentru cursa servomotoarelor. În codul Arduino s-au definit unghiuri maxime și minime de rotație absolute, prevenind astfel autodistrugerea mecanică a ansamblului, indiferent de valorile citite de senzori.

5.3. Sensibilitatea excesivă a senzorilor LDR (Jitter mecanic)

Problema: Fotorezistențele s-au dovedit a fi mult prea sensibile la cele mai mici variații de lumină. Acest lucru provoca un tremur constant și violent (trepidații) al panourilor fotovoltaice, deoarece sistemul încerca neîncetat să corecteze diferențe infime de iluminare.

Soluția: S-a adăugat o marjă de toleranță (histerezis) în algoritmul matematic. Microcontroller-ul ordonă mișcarea servomotoarelor doar atunci când diferența de lumină dintre senzori depășește un prag critic prestabilit, stabilizând complet structura la lumina ambientală.

5.4. Erori de execuție a comenzilor în MIT App Inventor

Problema: Comenzile trimise din aplicația mobilă către Arduino nu erau executate mereu. Aceasta era cauzată de formatarea imprecizabilă a datelor (asistentul Google adăuga automat

spații goale sau folosea diacritice) și de citirea desincronizată a caracterelor de pe modulul Bluetooth.

Soluția: S-a rescris logica de procesare a textului în App Inventor. Toate comenzile vocale sunt acum convertite în litere mici (downcase), iar aplicația caută doar anumite fragmente cheie (ex. fragmentul "nchide" pentru a evita conflictul între "inchide" și "închide"). De asemenea, s-a implementat o verificare a buffer-ului Bluetooth pentru a citi doar pachete de date complete.

6. CONCLUZII ȘI DIRECȚII VIITOARE DE DEZVOLTARE

6.1. Concluzii Generale Proiectul de față demonstrează cu succes integrarea complexă dintre o arhitectură hardware bazată pe senzori multipli și o interfață software mobilă, creând un sistem automatizat, sigur și eficient energetic. Obiectivele inițiale au fost atinse integral: mecanismul de urmărire solară (Tracker Pan-Tilt) optimizează captarea energiei, sistemul de management (BMS și Buck) asigură o funcționare autonomă în siguranță, iar bariera de acces răspunde prompt la comenzile vocale și la senzorii de proximitate, evitând accidentele. Comunicarea bidirecțională prin Bluetooth și aplicația Android (HMI) oferă utilizatorului un control total și o monitorizare a telemetriei în timp real.

6.2. Îmbunătățiri Propuse (Direcții Viitoare) Deși prototipul actual este complet funcțional și stabil, arhitectura sa modulară permite o serie de upgrade-uri viitoare pentru a-l transforma într-un sistem industrial complet:

Integrare IoT (Internet of Things): Înlocuirea modulului Bluetooth cu un modul Wi-Fi (ex. ESP8266 sau trecerea la un microcontroller ESP32). Acest lucru ar permite controlul barierei și monitorizarea panourilor solare de oriunde din lume, nu doar din proximitate.

Senzor de Vânt (Anemometru): Adăugarea unui senzor care să măsoare viteza vântului. În caz de furtună, sistemul ar putea ordona automat panourilor să se așeze în poziție orizontală (poziție de siguranță) pentru a evita distrugerea mecanică.

Tracking Astronomic (RTC/GPS): Pe lângă senzorii de lumină (LDR), integrarea unui modul de Timp Real (RTC) sau GPS ar permite calcularea matematică a poziției soarelui pe cer, utilă în zilele extrem de înnoirate când fotorezistențele nu pot determina un punct luminos clar.

Proiectarea unui Cablaj Imprimat (PCB): Trecerea de la fire tip jumper la proiectarea și fabricarea unei plăci electronice dedicate (PCB), ceea ce ar reduce volumul ocupat de componente și ar elimina definitiv riscul firelor imperfect conectate.

6.3. Bibliografie și Webografie

1. Documentația Oficială Arduino: <https://www.arduino.cc/reference/en/>

2. *Ghidul de Referință MIT App Inventor*: <http://ai2.appinventor.mit.edu/reference/>
3. *Datasheet Microcontroller ATmega2560*: Documentația tehnică a arhitecturii pe 8 biți.
4. *Datasheet Senzor Ultrasonic HC-SR04*: Specificații privind calculul timpului de zbor acustic.
5. *Principii de funcționare a sistemelor BMS (Battery Management System)* pentru celule Li-Ion.

ANEXA 1 – CODUL SURSĂ ARDUINO (C/C++)

```
#include <Servo.h> // Aducem în program fișierul (biblioteca) care știe
cum să miște servomotoarele

#include <Wire.h> // Aducem biblioteca necesară pentru comunicarea cu piese
complexe
#include <Adafruit_GFX.h> // Aducem biblioteca care ne ajută să desenăm forme și
text pe ecran
#include <Adafruit_SSD1306.h> // Aducem biblioteca specifică exact pentru modelul
nostru de ecran OLED

// SETĂRI PENTRU ECRANUL OLED
const int SCREEN_WIDTH = 128; // Îi spunem programului că ecranul are o lățime de
128 de pixeli
const int SCREEN_HEIGHT = 64; // Îi spunem că ecranul are o înălțime de 64 de
pixeli
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1); // Creăm
"obiectul" ecran cu setările de mai sus pentru a-l putea folosi

// SETĂRI PENTRU BARIERĂ
Servo bariera; // Creăm o "poreclă" (un obiect) numită "bariera" pentru a
controla motorașul acesteia
const int pinPIR = 2; // Senzorul de mișcare pentru mașini este legat la pinul
digital 2
const int pinServo = 9; // Cablul de control al motorașului barierei este legat
la pinul 9
const int pinLED = 13; // LED-ul care ne arată în ce mod suntem este legat la
pinul 13

// SETĂRI PENTRU SENZORUL DE OBSTACOLE (ULTRASONIC) ȘI ALARMĂ
const int pinTrig = 3; // Pinul care "strigă" (emite sunetul) este legat la 3
const int pinEcho = 4; // Pinul care "ascultă" (primește ecoul) este legat la 4
const int pinBuzzer = 5; // Difuzorul (alarma sonoră) este legat la pinul 5

// SETĂRI PENTRU PANOURILE SOLARE (TRACKER)
Servo servoOrizontal; // Creăm motorașul care mișcă panoul stânga-dreapta
Servo servoVertical; // Creăm motorașul care mișcă panoul sus-jos
const int pinServoOrizontal = 6; // Motorașul stânga-dreapta e legat la pinul 6
const int pinServoVertical = 7; // Motorașul sus-jos e legat la pinul 7

// SETĂRI PENTRU BUTOANE ȘI CITIREA BATERIEI
const int pinTouch = 8; // Senzorul pe care îl atingem cu degetul e la pinul 8
const int pinJoyX = A4; // Mișcarea stânga-dreapta a joystick-ului se citește la
pinul analogic A4
```

```

const int pinJoyY = A5; // Mișcarea sus-jos a joystick-ului se citește la pinul
analogic A5
const int pinJoySW = 10; // Butonul care se apasă când apăsăm pe joystick e la
pinul 10
const int pinVoltaj = A7; // Senzorul care măsoară curentul din baterie/panou e
la pinul A7
const int pinReleu = 22; // Comutatorul (releul) care schimbă ce măsurăm e la
pinul 22

// CORECȚIE PENTRU VOLTAJ
float factorCalibrare = 4.95; // Un număr matematic pentru a repara mica eroare a
senzorului de voltaj

// SETĂRI PENTRU SENZORII DE LUMINĂ (FOTOREZISTENȚE)
const int ldrStangaSus = A0; // Senzorul de lumină din stânga-sus e la pinul A0
const int ldrDreaptaSus = A1; // Senzorul de lumină din dreapta-sus e la pinul A1
const int ldrStangaJos = A2; // Senzorul de lumină din stânga-jos e la pinul A2
const int ldrDreaptaJos = A3; // Senzorul de lumină din dreapta-jos e la pinul A3

// SETĂRI PENTRU A EVITA TREMURATUL PANOURILOR
int tolerantaLumina = 25; // Panoul se va mișca doar dacă diferența de lumină e
mai mare de 25
int pasMiscare = 2; // Când se mișcă, motorușul face pași mici de câte 2 grade o
dată
int intervalMiscare = 50; // Face o pauză de 50 de milisecunde între pași ca să
nu fie brusc

// POZIȚIA DE PORNIRE A PANOURILOR
int pozitieOrizontala = 90; // Panoul pornește privind drept înainte (unghiul 90)
int pozitieVerticala = 90; // Panoul pornește la jumătatea distanței sus-jos
(unghiul 90)

// LIMITE DE SIGURANȚĂ CA SĂ NU SE RUPĂ FIRELE
const int limitaMinOrizantal = 10; // Panoul nu are voie să se rotească mai mult
de unghiul 10 în stânga
const int limitaMaxOrizantal = 170; // Panoul nu are voie să se rotească mai mult
de 170 în dreapta
const int limitaMinVertical = 20; // Panoul nu are voie să se lase prea mult în
jos (minim 20)
const int limitaMaxVertical = 135; // Panoul nu are voie să se dea prea mult pe
spate (maxim 135)

// REGULI PENTRU ALARMĂ (BUZZER)
const int BUZZER_OFF = HIGH; // Alarma tace când primește comanda HIGH (așa e
construită piesa)

```

```

const int BUZZER_ON = LOW; // Alarma sună când primește comanda LOW

// VARIABILE PENTRU A ȚINE MINTE LUCRURI
char comanda; // Aici vom salva literele pe care le primim de la telefon
(Bluetooth)
unsigned long timerBariera = 0; // Un ceasornic invizibil care numără când s-a
deschis bariera
unsigned long timerRadar = 0; // Un ceasornic care ne spune când am măsurat
ultima oară obstacolele
unsigned long timerSolar = 0; // Un ceasornic care ne spune când am mișcat ultima
oară panoul
unsigned long timerJoystick = 0; // Un ceasornic pentru a nu citi joystick-ul
prea des
unsigned long timerVoltaj = 0; // Un ceasornic pentru a afișa voltajul doar din
când în când

bool autoBarieraActiva = false; // Ține minte dacă bariera s-a deschis singură
(pentru a o și închide singură)
int ultimaCitirePIR = LOW; // Ține minte dacă înainte era sau nu cineva la
senzorul de mișcare
int pozitieBariera = 25; // Ține minte că unghiul 25 înseamnă că bariera este
închisă (lăsată jos)

bool modManual = false; // Dacă e 'false', soarele mișcă panourile. Dacă e
'true', le mișcăm noi.
bool ultimaStareButonJoy = HIGH; // Ține minte dacă butonul joystick-ului a fost
apăsât înainte
bool citirePanou = false; // Ne spune la ce ne uităm: la bateria generală sau la
curentul din panouri
int ultimaStareTouch = LOW; // Ține minte dacă am atins senzorul cu degetul
înainte

// FUNCȚIA DE PORNIRE (Rulează o singură dată când băgăm sistemul în priză)
void setup() {
    pinMode(pinPIR, INPUT); // Senzorul de mișcare doar ne trimite date (este
intrare)
    pinMode(pinLED, OUTPUT); // LED-ul primește curent de la noi ca să se aprindă
(este ieșire)
    pinMode(pinTrig, OUTPUT); // Pinul care strigă trimite sunetul (ieșire)
    pinMode(pinEcho, INPUT); // Pinul care ascultă primește ecoul (intrare)
    pinMode(pinBuzzer, OUTPUT); // Difuzorul primește curent ca să sune (ieșire)
    pinMode(ldrStangaSus, INPUT); // Toți cei 4 senzori de lumină primesc lumina de
afară (sunt intrări)
    pinMode(ldrDreaptaSus, INPUT);
    pinMode(ldrStangaJos, INPUT);

```

```

    pinMode(ldrDreaptaJos, INPUT);
    pinMode(pinTouch, INPUT); // Senzorul de atingere simte degetul (intrare)
    pinMode(pinJoySW, INPUT_PULLUP); // Butonul joystick este setat special să nu
    aibă erori de la paraziti electrici
    pinMode(pinReleu, OUTPUT); // Releul primește comenzi de la noi să se cupleze
    (ieșire)

    digitalWrite(pinReleu, LOW); // La pornire, îi spunem releului să stea decuplat
    Serial.begin(9600); // Pornim comunicarea cu calculatorul la viteza 9600 pentru
    a vedea mesaje
    Serial1.begin(9600); // Pornim comunicarea cu modulul Bluetooth la aceeași
    viteză

    // PORNIREA ECRANULUI OLED
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // Dacă ecranul nu este găsit
    la adresa 0x3C...
        Serial.println(F("Eroare la pornirea OLED-ului")); // ...scrie pe calculator
    că e o eroare
    } else { // Altfel, dacă ecranul a fost găsit cu succes...
        display.clearDisplay(); // Curățăm ecranul de orice mizerie anterioară
        display.setTextSize(2); // Facem textul mare (mărimea 2)
        display.setTextColor(WHITE); // Setăm culoarea textului pe alb (sau luminos)
        display.setCursor(0, 10); // Punem stiloul imaginar la coordonatele 0 pe
    lățime și 10 pe înălțime
        display.println(" SISTEM "); // Scriem cuvântul "SISTEM"
        display.setCursor(0, 30); // Mutăm stiloul mai jos
        display.println(" PORNIT "); // Scriem cuvântul "PORNIT"
        display.display(); // Dăm comanda efectivă să apară textul pe ecran
        delay(2000); // Lăsăm textul acolo 2 secunde ca să poată fi citit
    }

    bariera.attach(pinServo); // Conectăm logic motorășul barierei la pinul lui
    bariera.write(25); // Îl obligăm să se ducă la unghiul 25 (să stea închis la
    pornire)
    delay(1000); // Îi dăm 1 secundă ca să aibă timp să ajungă acolo
    bariera.detach(); // Îi tăiem curentul de control ca să nu bazaie sau să
    consume degeaba

    digitalWrite(pinLED, LOW); // Ne asigurăm că LED-ul este stins la pornire
    digitalWrite(pinBuzzer, BUZZER_OFF); // Ne asigurăm că alarma este mută

    servoOrizantal.attach(pinServoOrizantal); // Conectăm logic motorășele panoului
    servoVertical.attach(pinServoVertical);
    servoOrizantal.write(pozitieOrizantala); // Le trimitem la poziția de mijloc
    (drept înainte)

```

```

servoVertical.write(pozitieVerticala);

Serial.println("Sistem pornit! OLED si Releu activate."); // Trimitem un mesaj
de succes pe calculator
}
// FUNCȚIE CARE MĂSOARĂ DISTANȚA CU SUNET
int masoaraDistanța() {
    digitalWrite(pinTrig, LOW); // Ne asigurăm că boxa e oprită
    delayMicroseconds(2); // Așteptăm foarte puțin
    digitalWrite(pinTrig, HIGH); // Dăm drumul la un sunet scurt (țiuit invizibil)
    delayMicroseconds(10); // Îl lăsăm să sune 10 microsecunde
    digitalWrite(pinTrig, LOW); // Oprim sunetul

    long durata = pulseIn(pinEcho, HIGH, 30000); // Măsurăm cât timp îi ia
    sunetului să lovească obstacolul și să se întoarcă
    if (durata == 0) { // Dacă sunetul nu s-a mai întors (s-a pierdut în aer)...
        return 999; // ...înseamnă că nu e niciun obstacol, deci dăm o distanță falsă
        foarte mare (999 cm)
    }
    return durata * 0.034 / 2; // Înmulțim timpul cu viteza sunetului și împărțim
    la 2 (deoarece sunetul a făcut drum dus-întors)
}

// FUNCȚIE CARE CAUTĂ SOARELE SINGURĂ
void urmaresteSoarele() {
    int valStangaSus = analogRead(ldrStangaSus); // Citim câtă lumină lovește
    colțul stânga-sus
    int valDreaptaSus = analogRead(ldrDreaptaSus); // Citim lumina din dreapta-sus
    int valStangaJos = analogRead(ldrStangaJos); // Citim lumina din stânga-jos
    int valDreaptaJos = analogRead(ldrDreaptaJos); // Citim lumina din dreapta-jos

    int mediaSus = (valStangaSus + valDreaptaSus) / 2; // Calculăm media de lumină
    pentru toată partea de sus a panoului
    int mediaJos = (valStangaJos + valDreaptaJos) / 2; // Calculăm media pentru
    partea de jos
    int mediaStanga = (valStangaSus + valStangaJos) / 2; // Calculăm media pentru
    jumătatea din stânga
    int mediaDreapta = (valDreaptaSus + valDreaptaJos) / 2; // Calculăm media
    pentru jumătatea din dreapta

    int diferentaVerticala = mediaSus - mediaJos; // Vedem care e diferența între
    sus și jos
    int diferentaOrizontala = mediaStanga - mediaDreapta; // Vedem diferența între
    stânga și dreapta

```



```

    bool aMiscatVertical = false; // O variabilă care ține minte dacă s-a făcut
    vreo mișcare sus-jos

    if (abs(diferentaVerticala) > tolerantaLumina) { // Dacă diferența este destul
    de mare încât să merite mișcarea...
        if (mediaSus > mediaJos && pozitieVerticala > limitaMinVertical) { // Dacă e
        mai mult soare sus și nu s-a blocat panoul...
            pozitieVerticala -= pasMiscare; // Ridicăm unghiul cu câțiva pași mici
            aMiscatVertical = true; // Notăm că tocmai s-a mișcat
        } else if (mediaSus < mediaJos && pozitieVerticala < limitaMaxVertical) { //
        Dar dacă e mai mult soare jos...
            pozitieVerticala += pasMiscare; // Lăsăm unghiul mai în jos
            aMiscatVertical = true; // Notăm că s-a mișcat
        }
        if (aMiscatVertical) { // Dacă s-a decis o mișcare din calculele de mai
        sus...
            servoVertical.write(pozitieVerticala); // Trimitem efectiv comanda la
            motoraș
            delay(15); // Îi dăm o fracțiune de secundă să facă mișcarea ca să nu
            tremure
        }
    }

    if (abs(diferentaOrizontala) > tolerantaLumina) { // Dacă diferența stânga-
    dreapta e suficient de mare...
        bool aMiscatOrizontal = false; // Ținem minte dacă va face vreo mișcare
        laterală
        if (mediaStanga > mediaDreapta && pozitieOrizontala > limitaMinOrizontal) {
        // Dacă soarele e în stânga...
            pozitieOrizontala -= pasMiscare; // Rotim motorașul în stânga
            aMiscatOrizontal = true; // Notăm mișcarea
        } else if (mediaStanga < mediaDreapta && pozitieOrizontala <
        limitaMaxOrizontal) { // Dacă soarele e în dreapta...
            pozitieOrizontala += pasMiscare; // Rotim motorașul în dreapta
            aMiscatOrizontal = true; // Notăm mișcarea
        }
        if (aMiscatOrizontal) { // Dacă s-a mișcat pe orizontală...
            servoOrizontal.write(pozitieOrizontala); // Trimitem comanda la motorașul
            de bază
        }
    }
}

// FUNCȚIE CARE MIȘCĂ PANOUL DIN MANETĂ (MANUAL)
void miscareDinJoystick() {

```

```

    int valX = analogRead(pinJoyX); // Citim în ce direcție e tras joystick-ul
    stânga-dreapta
    int valY = analogRead(pinJoyY); // Citim direcția sus-jos

    if (valX < 400 && pozitieOrizontala < limitaMaxOrizontal) { // Dacă maneta e
    trasă la stânga...
        pozitieOrizontala += pasMiscare; // Rotim panoul în stânga
    }
    if (valX > 600 && pozitieOrizontala > limitaMinOrizontal) { // Dacă maneta e
    trasă la dreapta...
        pozitieOrizontala -= pasMiscare; // Rotim panoul în dreapta
    }
    if (valY < 400 && pozitieVerticala > limitaMinVertical) { // Dacă maneta e
    împinsă în sus...
        pozitieVerticala -= pasMiscare; // Ridicăm panoul
    }
    if (valY > 600 && pozitieVerticala < limitaMaxVertical) { // Dacă maneta e
    trasă în jos...
        pozitieVerticala += pasMiscare; // Lăsăm panoul în jos
    }

    servoOrizontal.write(pozitieOrizontala); // Aplicăm unghiurile calculate pe
    motorașul orizontal
    servoVertical.write(pozitieVerticala); // Aplicăm unghiurile pe motorașul
    vertical
}

// BUCLA PRINCIPALĂ (Rulează la infinit cât timp sistemul are curent)
void loop() {
    unsigned long timpulCurent = millis(); // Ne uităm la ceas să vedem câte
    milisekunde au trecut de la pornire

    // CITIREA SENZORULUI TACTIL (PENTRU RELEU)
    int stareTouchCurenta = digitalRead(pinTouch); // Verificăm dacă cineva ține
    degetul pe senzor
    if (stareTouchCurenta == HIGH && ultimaStareTouch == LOW) { // Dacă abia l-a
    apăsât (înainte nu era apăsât)...
        citirePanou = !citirePanou; // Schimbăm variabila (dacă era Baterie, devine
    Panou, și invers)
        if (citirePanou) { // Dacă acum trebuie să citim bateria generală...
            digitalWrite(pinReleu, HIGH); // Pâlpâim releul (îl activăm)
        } else { // Dacă trebuie să citim din panoul solar...
            digitalWrite(pinReleu, LOW); // Închidem releul
        }
    }
}

```

```

    delay(200); // Așteptăm o fracțiune de secundă ca să nu citească o atingere
de 10 ori
}
ultimaStareTouch = stareTouchCurenta; // Salvăm starea pentru data viitoare

// MĂSURAREA ȘI AFIȘAREA VOLTAJULUI PE ECRAN ȘI TELEFON
if (timpulCurent - timerVoltaj >= 2000) { // Doar o dată la fiecare 2 secunde
facem asta...
    int valoareAnalogica = analogRead(pinVoltaj); // Citim valoarea electrică
brută din senzor (numere între 0 și 1023)
    float voltaj = (valoareAnalogica * 5.0) / 1024.0 * factorCalibrare; //
Transformăm numărul brut în volți reali prin matematică

    display.clearDisplay(); // Ștergem ecranul ca să scriem noile valori
    display.setTextSize(2); // Setăm mărimea textului
    display.setCursor(0, 0); // Ne ducem sus de tot în stânga

    if (citirePanou) { // Dacă releul măsoară bateria...
        Serial.print("Voltaj Baterie: "); // Anunțăm pe calculator
        display.println("BATERIE:"); // Scriem "BATERIE" pe ecran
        Serial1.print("B"); // Trimitem litera B pe Bluetooth ca aplicația să știe
ce primește
        Serial1.println(voltaj); // Trimitem numărul de volți pe Bluetooth
    } else { // Dacă releul e setat pe panoul solar...
        Serial.print("Voltaj Panou: "); // Anunțăm pe calculator
        display.println("PANOU SOL:"); // Scriem "PANOU SOL" pe ecran
        Serial1.print("P"); // Trimitem litera P prin Bluetooth pentru aplicație
        Serial1.println(voltaj); // Trimitem numărul de volți pe Bluetooth
    }

    Serial.print(voltaj); // Trimitem și pe cablul de la calculator
    Serial.println(" V"); // Punem și litera V după el

    display.setCursor(0, 30); // Ne ducem mai jos pe ecranul OLED
    display.setTextSize(3); // Facem cifrele foarte mari
    display.print(voltaj, 1); // Scriem voltajul cu o singură virgulă (ex: 12.4)
    display.println("V"); // Desenăm litera V
    display.display(); // Dăm comanda ca ecranul să se aprindă cu noul text

    timerVoltaj = timpulCurent; // Resetăm ceasornicul ca să mai aștepte 2
secunde până la următoarea citire
}

// SCHIMBAREA MODULUI DIN BUTONUL DE LA JOYSTICK

```

```

    bool stareButonCurenta = digitalRead(pinJoySW); // Citim dacă e apăsat butonul
joystick-ului
    if (stareButonCurenta == LOW && ultimaStareButonJoy == HIGH) { // Dacă abia a
fost apăsat ferm...
        modManual = !modManual; // Trecem din modul automat (soare) în manual, sau
invers
        if (modManual) { // Dacă am trecut pe manual...
            Serial.println(">>> MOD MANUAL (JOYSTICK) ACTIVAT <<<"); // Anunțăm pe
calculator
            digitalWrite(pinLED, HIGH); // Aprindem becul roșu de martor
        } else { // Dacă am trecut pe automat...
            Serial.println(">>> MOD AUTOMAT (SOARE) ACTIVAT <<<"); // Anunțăm pe
calculator
            digitalWrite(pinLED, LOW); // Stingem becul roșu
        }
        delay(200); // Așteptăm scurt ca să nu citească butonul de două ori
    }
    ultimaStareButonJoy = stareButonCurenta; // Ținem minte că am apăsat butonul

// DECIZIA: MIȘCĂM PANOUL MANUAL SAU DUPĂ SOARE?
    if (modManual == true) { // Dacă suntem pe modul manual...
        if (timpulCurent - timerJoystick >= 30) { // Și au trecut 30 de
milisecunde...
            miscareDinJoystick(); // Chemăm funcția care mișcă din joystick
            timerJoystick = timpulCurent; // Resetăm ceasornicul joystick-ului
        }
    } else { // Dacă suntem pe automat...
        if (timpulCurent - timerSolar >= intervalMiscare) { // Și a trecut suficient
timp de pauză...
            urmaresteSoarele(); // Chemăm funcția care citește lumina și urmărește
soarele
            timerSolar = timpulCurent; // Resetăm ceasornicul solar
        }
    }
}

// SENZORUL CARE "ASCULTĂ" DACĂ SUNT OBSTACOLE
    int citirePIR_Curenta = digitalRead(pinPIR); // Citim senzorul care detectează
mișcarea (pentru mașini)
    if (timpulCurent - timerRadar >= 2000) { // La fiecare 2 secunde...
        int d = masoaraDistanta(); // Măsurăm în fundal dacă există ceva sub barieră
(pentru siguranță)
        timerRadar = timpulCurent; // Resetăm ceasornicul radarului
    }

// INTERPRETAREA COMENZILOR DE LA TELEFON (BLUETOOTH)

```

```

    if (Serial1.available() > 0) { // Dacă telefonul a trimis un mesaj spre noi...
        comanda = Serial1.read(); // Îl citim și îl băgăm în cutiuța cu litera
        'comanda'
        if (comanda != '\n' && comanda != '\r' && comanda != ' ') { // Ignorăm
            caracterele goale sau de rând nou

            if (comanda == 'X' || comanda == 'x' || comanda == '1') { // Dacă am primit
                X sau 1 de la aplicație...
                autoBarieraActiva = false; // Trecem peste senzorul automat (am cerut noi
                deschiderea)
                deschideBariera(); // Ridicăm bariera
            } else if (comanda == 'Y' || comanda == 'y') { // Dacă am primit Y
                (închide)...
                if (masoaraDistanța() < 15) { // Verificăm rapid dacă mașina e încă sub
                braț (sub 15 cm)
                    eroareBuzzer(); // Dacă este, refuzăm închiderea și dăm un sunet de
                    eroare
                } else { // Dacă mașina a trecut și e liber...
                    autoBarieraActiva = false; // Trecem peste senzor
                    inchideBariera(); // Lăsăm bariera jos
                }
            } else if (comanda == 'M' || comanda == 'm') { // Dacă am primit litera M
                de la butonul aplicației...
                modManual = !modManual; // Schimbăm modul panourilor (Automat <-> Manual)
                if (modManual) {
                    Serial.println(">>> MOD MANUAL (APP) ACTIVAT <<<");
                    digitalWrite(pinLED, HIGH); // Aprindem led-ul martor
                } else {
                    Serial.println(">>> MOD AUTOMAT (APP) ACTIVAT <<<");
                    digitalWrite(pinLED, LOW); // Stingem led-ul martor
                }
            } else if ((comanda == 'U' || comanda == 'u') && modManual == true) { //
                Dacă telefonul a trimis U (Up - Sus)
                if (pozitieVerticala > limitaMinVertical) { // Verificăm să nu forțăm
                prea tare mecanismul
                    pozitieVerticala -= pasMiscare; // Ridicăm panoul
                }
                servoVertical.write(pozitieVerticala); // Trimitem motorașul
            } else if ((comanda == 'D' || comanda == 'd') && modManual == true) { //
                Dacă am primit D (Down - Jos)
                if (pozitieVerticala < limitaMaxVertical) {
                    pozitieVerticala += pasMiscare; // Lăsăm panoul în jos
                }
                servoVertical.write(pozitieVerticala);
            }
        }
    }
}

```

```

    } else if ((comanda == 'L' || comanda == 'l') && modManual == true) { //
Dacă am primit L (Left - Stânga)
        if (pozitieOrizontala < limitaMaxOrizontala) {
            pozitieOrizontala += pasMiscare; // Îl dăm stânga
        }
        servoOrizontal.write(pozitieOrizontala);
    } else if ((comanda == 'R' || comanda == 'r') && modManual == true) { //
Dacă am primit R (Right - Dreapta)
        if (pozitieOrizontala > limitaMinOrizontala) {
            pozitieOrizontala -= pasMiscare; // Îl dăm în dreapta
        }
        servoOrizontal.write(pozitieOrizontala);
    }
}
}

// DESCHIDEREA AUTOMATĂ A BARRIEREI
if (citirePIR_Curenta == HIGH && ultimaCitirePIR == LOW) { // Dacă senzorul
vede o mașină care se apropie
    autoBarieraActiva = true; // Notăm că bariera s-a deschis singură
    deschideBariera(); // Ridicăm brațul
    timerBariera = timpulCurent; // Începem să numărăm secundele
}
ultimaCitirePIR = citirePIR_Curenta; // Salvăm starea ca să nu ridicăm bariera
de două ori la rând

// ÎNCHIDEREA AUTOMATĂ A BARRIEREI
if (autoBarieraActiva && (timpulCurent - timerBariera >= 30000)) { // Dacă
bariera s-a deschis singură și au trecut 30 de secunde (timp destul de ieșire)
    if (masoaraDistanta() < 15) { // Verificăm dacă mașina cumva a adormit sub
barieră
        timerBariera = timpulCurent - 25000; // Dacă da, păcălim ceasul scăzând 25
de sec (mai așteaptă 5 secunde și verifică iar)
    } else { // Dacă zona e curată...
        inchideBariera(); // Coborâm brațul
        autoBarieraActiva = false; // Resetăm starea
    }
}
}

// FUNCȚIILE CARE MIȘCĂ EFECTIV BARIERA
void deschideBariera() {
    bariera.attach(pinServo); // Conectăm electric motorușul ca să aibă putere
    digitalWrite(pinBuzzer, BUZZER_ON); // Dăm drumul la alarmă cât timp brațul se
mișcă

```

```

    for (int i = pozitieBariera; i <= 101; i++) { // Facem un șirag de pași,
    mergând unghi cu unghi spre cer (maxim 101)
        bariera.write(i); // Trimitem câte un grad pe rând la motorăș
        delay(15); // Așteptăm extrem de puțin între grade pentru ca mișcarea să fie
    lentă și grațioasă
    }
    pozitieBariera = 101; // Notăm că bariera a ajuns sus de tot
    digitalWrite(pinBuzzer, BUZZER_OFF); // Oprim alarma pentru că mișcarea s-a
    terminat
}

void inchideBariera() {
    for (int i = 0; i < 3; i++) { // Înainte să cadă, facem alarma să sune de 3 ori
    (beep, beep, beep)
        digitalWrite(pinBuzzer, BUZZER_ON); // Aprindem difuzorul
        delay(150); // Îl ținem aprins
        digitalWrite(pinBuzzer, BUZZER_OFF); // Stingem difuzorul
        delay(150); // Îl ținem stins
    }
    digitalWrite(pinBuzzer, BUZZER_ON); // Aprindem difuzorul continuu în timp ce
    cade brațul
    for (int i = pozitieBariera; i >= 25; i--) { // Numărăm înapoi de sus (101)
    până jos (25)
        bariera.write(i); // O lăsăm un grad mai jos
        delay(20); // Așteptăm 20ms ca să cadă blând și să nu se lovească tare
    }
    pozitieBariera = 25; // Salvăm poziția ca fiind "închisă"
    digitalWrite(pinBuzzer, BUZZER_OFF); // Oprim alarma
    delay(200); // Îi lăsăm 200 de milisecunde ca brațul să nu mai tremure fizic de
    la șoc
    bariera.detach(); // Îi tăiem curentul electric ca să nu stea sub tensiune
    (protejează plasticul din interior)
}

// FUNCȚIE PENTRU SUNET DE EROARE
void eroareBuzzer() {
    digitalWrite(pinBuzzer, BUZZER_ON); // Dăm drumul la alarmă
    delay(800); // O lăsăm să scoată un sunet lung (BEEEEEP) aproape o secundă
    digitalWrite(pinBuzzer, BUZZER_OFF); // O oprim
}

```